# Índices en Bases de Datos MySQL

## 1. Introducción

En una base de datos, los índices son estructuras auxiliares que mejoran la velocidad de búsqueda de datos. Su correspondiente índice es un el índice en el libro, que permite encontrar información sin leer todas las páginas. Sin índices, MySQL debe realizar un escaneo completo de la tabla.

Cuando se crea una tabla, MySQL automáticamente crea un índice para la clave primaria (PRIMARY KEY). Los índices secundarios pueden crearse sobre columnas específicas para acelerar consultas frecuentes, pero ocupan espacio en disco y ralentizan escrituras (INSERT, UPDATE, DELETE).

## 2. Concepto básico

La forma en MySQL es: cuando se ejecuta una consulta, el motor verifica si hay un índice en la cláusula WHERE. Si encuentra un índice útil, el motor lee ese índice para localizar las filas exactas en lugar de escanear todas las filas. Los índices se almacenan en estructuras de datos ordenadas (típicamente B-tree) que permiten búsquedas rápidas.

```
SELECT * FROM tabla WHERE columna = 'valor';
```

## ¿Por qué usar índices?

**Ventajas principales:**

- **Velocidad de búsqueda**: Reduce complejidad de O(n) a O(log n)
- **Eficiencia en la ejecución** del motor de consultas: Permite operaciones como WHERE, ORDER BY, JOIN, GROUP BY
- **Ordenamiento rápido**: Evita ordenamientos en memoria cuando el índice ya está ordenado
- **Optimización de JOIN**: Acelera combinaciones de tablas
- **Unicidad garantizada**: Índices UNIQUE evitan duplicados

**Desventajas a tener en cuenta:**

- Ocupan espacio adicional en disco (índices)
- Ralentizan escrituras (INSERT, UPDATE, DELETE) porque el sistema debe también actualizar los índices
- Requieren mantenimiento periódico (ANALYZE TABLE, OPTIMIZE TABLE)
- Pueden confundir al optimizador si son demasiados o innecesarios

**Regla de oro:** Crea índices en columnas que aparezcan frecuentemente en WHERE, JOIN, ORDER BY o GROUP BY, pero evita crear demasiados porque ralentizan las escrituras.

## 3. Índices B-Tree

Los índices B-Tree son el tipo predeterminado. Son estructuras de datos ordenadas que permiten búsquedas rápidas en rangos.

### ¿Cómo funciona?

El árbol está organizado en niveles: hoja, rama, raíz. Cada nodo **hoja** contiene **punteros** a filas reales. Cada nodo **rama** contiene rangos que dirigen la búsqueda hacia los nodos hoja correctos. Los nodos **raíz** son el punto de entrada.

Ejemplo: Buscar id = 5 en un índice con raíz [3, 7] y hojas [1, 2] [3, 4, 5] [7, 8, 9]:

1. Raíz: 5 > 3 y 5 < 7 → ir a la hoja central
2. Hoja: encontrar 5 → obtener puntero a la fila

Complejidad: O(log n) para búsqueda, inserción, eliminación. Mucho más rápido que escaneo completo O(n).

### Características

- Estructura ordenada jerárquica
- Soporta operaciones rango: >, <, >=, <=, BETWEEN
- Soporta ORDER BY sin ordenamiento
- Los datos están ordenados en las hojas

### Limitaciones

- No funcionamiento bien ante cambios frecuentes, actualización y eliminación masiva
- No recomendado si el 50% o más de la tabla cambia

### Representación simplificada de un B-Tree

Leyenda: Raíz (nivel 0) → Ramas (nivel 1) → Hojas (nivel 2). Las hojas contienen los datos ordenados.

## 4. Índice Clustered (Agrupado) vs Non-Clustered (No Agrupado)

### Índice Clustered (Agrupado)

- Determina cómo se almacenan los datos en la tabla
- Solo puede haber uno por tabla
- Típicamente la clave primaria es el agrupado por defecto
- La tabla ocupa el orden del índice de la clave primaria

### ¿Por qué es grande?

Un índice B-Tree puede tener muchos niveles si la tabla es grande. Cada nodo puede tener miles de filas. Por eso hay que limitar el número agregado cada columna/expresión a 16 bytes máximo. Si la clave es muy grande, MySQL puede rechazar la creación o ralentizar operaciones.

### Índice Non-Clustered (No Agrupado)

- Es similar a un índice de libro
- Contiene punteros a las filas
- Puede haber muchos por tabla
- Ocupa espacio adicional en disco
- Ralentiza escrituras (INSERT, UPDATE, DELETE) porque hay que actualizar todos los índices afectados

### Importante: ¿qué pasa si no hay clave primaria?

Si no defines clave primaria, MySQL usará el primer índice secundario único como fallback. Si no hay ni eso, MySQL crea internamente un índice oculto.

## 5. Índices Compuestos

Un índice compuesto indexa múltiples columnas. Ejemplo: `CREATE INDEX idx_ventas_cliente_fecha ON ventas(cliente_id, fecha);`

### Regla del prefijo izquierdo

El índice es útil para consultas que usan las columnas en orden, desde la izquierda:

```
-- ✅ Usa el índice completo
WHERE cliente_id = 5 AND fecha > '2024-01-01'

-- ✅ Usa el índice (solo cliente_id)
WHERE cliente_id = 5

-- ❌ No usa el índice (falta cliente_id)
WHERE fecha > '2024-01-01'
```

### Orden importa en WHERE

Las condiciones pueden estar en cualquier orden dentro del WHERE, el optimizador reordena internamente.

### Cuándo no funciona

- Si omites la columna inicial
- Si usas OR en lugar de AND
- Si la columna está dentro de una función: `WHERE YEAR(fecha) = 2024` no usa índice de `(fecha)`

## 6. Índices Parciales

MySQL no soporta índices parciales nativos (a diferencia de PostgreSQL). Alternativas:

- Índice completo
- Columna generada + índice
- Particionamiento

## 7. Índices Funcionales

Desde MySQL 8.0.13 puedes indexar expresiones:

```
CREATE INDEX idx_email_lower ON clientes(LOWER(email));
```

Útil para búsquedas case-insensitive, cálculos matemáticos, extracción de fechas, etc.

## 8. Índices Invisibles

Permiten ocultar un índice al optimizador sin eliminarlo. Útil para probar el impacto de quitar un índice sin perder su definición.

```
ALTER TABLE tabla ALTER INDEX idx_nombre INVISIBLE;
ALTER TABLE tabla ALTER INDEX idx_nombre VISIBLE;
```

## 9. EXPLAIN: cómo saber si tu índice se usa realmente

`EXPLAIN` es una herramienta fundamental para analizar cómo MySQL piensa ejecutar una consulta. Muestra el plan de ejecución.

```
EXPLAIN SELECT * FROM clientes WHERE email = 'test@mail.com';
```

Salida típica:

```
+----+-------------+---------+------+---------------+------+---------+-------+------+-------+
| id | select_type | table   | type | possible_keys | key  | key_len | ref   | rows | Extra |
+----+-------------+---------+------+---------------+------+---------+-------+------+-------+
|  1 | SIMPLE      | clientes | ref  | idx_email     | idx_email | 255     | const |    1 | NULL  |
+----+-------------+---------+------+---------------+------+---------+-------+------+-------+
```

Interpretación: usa el índice idx_email, busca una fila exacta (const), examina 1 fila, sin operaciones adicionales.

### Columnas clave

| Columna | Qué significa |
|---|---|
| `type` | Cómo se accede a la tabla. Ideal: const, eq_ref, ref, range. Malo: ALL (escaneo completo) |
| `possible_keys` | Índices que podrían usarse |
| `key` | Índice que realmente se usa. Si es NULL, no hay índice |
| `key_len` | Bytes del índice usados. Útil para saber si usas toda la clave o solo parte |
| `rows` | Estimación de filas a examinar. Cuanto menos, mejor |
| `Extra` | Información adicional. Cuidado con: Using filesort, Using temporary |

### Valores de type (de mejor a peor)

system > const > eq_ref > ref > range > index > ALL

Regla práctica: apunta a que al menos veas range en tus consultas frecuentes.

## 10. EXPLAIN ANALYZE (MySQL 8.0+)

Ejecuta la consulta realmente y muestra tiempos reales:

```
EXPLAIN ANALYZE SELECT * FROM clientes WHERE email = 'test@mail.com';
```

Salida típica:

```
-> Single-row index lookup on clientes using idx_email (email='test@mail.com')  (cost=0.00 rows=1) (actual time=0.000..0.000 rows=1 loops=1)
```

Diferencia con EXPLAIN: muestra costo estimado vs tiempo real, número de filas estimadas vs reales, cuántas veces se ejecutó el nodo (loops).

## 11. Índices Full-Text (para búsqueda de texto)

Los índices B-Tree no sirven para buscar palabras dentro de texto. Para eso existen los índices FULLTEXT.

```
CREATE FULLTEXT INDEX idx_articulos_texto ON articulos(titulo, contenido);

SELECT * FROM articulos WHERE MATCH(titulo, contenido) AGAINST('base de datos');
```

Características:

- Búsqueda por palabras completas
- Ignora palabras comunes (stopwords)
- Soporta modos: natural language, boolean, query expansion
- Requiere tabla MyISAM o InnoDB (desde MySQL 5.6)

## 12. Índices Hash

Estructura: tabla hash en memoria.

Ventajas: búsqueda exacta ultra rápida O(1).

Limitaciones:

- Solo igualdad (=)
- No sirve para rangos, ORDER BY, LIKE
- Solo en memoria (MEMORY engine)

Casi nunca se usan en producción.

## 13. Cardinalidad

Cardinalidad = número de valores distintos en una columna.

Alta cardinalidad: email, id, timestamp → índice muy selectivo, ayuda mucho.

Baja cardinalidad: sexo, activo, estado → índice casi inútil, MySQL lo ignora.

Ver cardinalidad:

```
SHOW INDEX FROM tabla;
```

Columna Cardinality.

Actualizar estadísticas:

```
ANALYZE TABLE tabla;
```

## 14. Mantenimiento

Los índices se fragmentan con el tiempo. Síntomas: rendimiento degradado, espacio en disco crece sin datos nuevos.

Mantenimiento periódico:

```
OPTIMIZE TABLE tabla;
ANALYZE TABLE tabla;
```

Frecuencia: semanal o mensual en producción.

## 15. Buenas prácticas

- No crear índices innecesarios
- Monitorear uso de índices
- Considerar cardinalidad antes de indexar
- Usar EXPLAIN antes de asumir que funciona
- Mantener estadísticas actualizadas
- Backup antes de cambios masivos
- Documentar el propósito de cada índice

## 16. Preguntas guía (con respuesta)

1. ¿Qué pasa si tengo 10 índices en una tabla con 1 millón de filas?
   Ralentiza escrituras significativamente (cada INSERT actualiza 10 índices). Pero esas consultas tienen chance de usar un índice correcto.
2. ¿Por qué el orden importa?
   Porque el índice está ordenado según ese orden. Un índice en (a, b) sirve para ORDER BY a, b pero no para ORDER BY b, a.
3. ¿Cómo sé si mi índice se está usando?
   Con EXPLAIN y verificando que el campo key no sea NULL.
4. ¿Por qué mi consulta no usa el índice que creé?
   Posibles razones: cardinalidad baja, tabla pequeña (MySQL prefiere escaneo completo), estadísticas desactualizadas, función en la columna, tipo de dato incompatible.
5. ¿Qué es el "covering index"?
   Un índice que contiene todas las columnas que la consulta necesita, así MySQL no tiene que leer la tabla, solo el índice.
6. ¿Cuándo no crear índices?
   En tablas pequeñas, en columnas con baja cardinalidad, en tablas con escrituras masivas y pocas lecturas, en columnas que cambian mucho.

## 17. Casos

Los casos reales en empresas dan contexto práctico. Ejemplos típicos: búsqueda de usuarios por email, filtrado de pedidos por estado y fecha, JOIN entre tablas grandes, ORDER BY en columnas sin índice, GROUP BY en columnas sin índice.

Herramientas para detectar casos: slow query log, EXPLAIN en producción, monitoreo de conexiones activas, profiling con SHOW PROFILE.

## 18. Conclusión

Los índices son el arma más poderosa para optimizar consultas en MySQL. Pero también son una herramienta de doble filo: cada índice tiene costo en escrituras y espacio en disco.

El ciclo de vida: diseñar → crear → medir → ajustar → mantener.

Regla final: no adivines, mide. Usa EXPLAIN, monitorea slow queries, y decide con datos reales.

Material de estudio para DBA y desarrolladores
Contenido compilado por Claude con revisión y ejemplos por GitHub Copilot